

Deakin University
SIT333 Software Quality and Testing
Task 8.1P - STEM Game Web Application Testing

Submitted by: Ansh Sharma
Student ID: 225451575

1. Introduction

This task extends the 7.1P login web application into a STEM math game for grade 4-6 students. The application uses a Java Spring Boot web structure where JSP pages are served through Java endpoints. After successful login, the user is taken through math questions. Correct answers move the user to the next question, while incorrect or invalid inputs keep the user on the same page with an error message.

The submission includes developer-side unit testing for MathQuestionService.java and tester-side functional testing using Selenium. The focus is to verify math operation correctness, prevent error pages from invalid inputs, and test the integrated flow between JSP pages, RoutingServlet.java, and backend service logic.

2. Project Structure and Web Application Flow

File / Component	Purpose
MyServer.java	Starts the Spring Boot web application on port 8080.
RoutingServlet.java	Maps GET and POST URLs such as /login, /q1, /q2 and /q3 to Java handler methods.
LoginService.java	Authenticates the user before the STEM game starts.
MathQuestionService.java	Contains addition, subtraction and multiplication functions used by the question pages.
view-login.jsp	Login page for username, password and date of birth.
view-q1.jsp	First STEM question page: addition.
view-q2.jsp	Second STEM question page: subtraction.
view-q3.jsp	Third STEM question page: multiplication.
TestMathQuestionService.java	JUnit unit tests for math operation logic.
StemGameFunctionalTest.java	Selenium functional tests for login and question navigation.

The flow of the application is: Login page -> Q1 addition -> Q2 subtraction -> Q3 multiplication -> completion/return page. This makes the application more integrated than 7.1P because JSP pages, routing, form submissions, redirect behaviour and backend math logic all work together in one web project.

3. Code Changes Made

File Changed	Update Made	Reason
LoginService.java	Updated login validation to accept username, password and DOB.	Ensures successful login routes the user to Q1.
MathQuestionService.java	Added safe input validation and q3Multiplication().	Prevents NumberFormatException and supports the third STEM question.
RoutingServlet.java	Updated q1/q2 POST handling and added q3 GET/POST routes.	Allows correct navigation and error messages for the STEM game.
view-q3.jsp	Created/updated the third question JSP page.	Meets task requirement to create a third math problem page.
TestMathQuestionService.java	Added unit tests for valid, empty and invalid inputs.	Tests math logic from developer point of view.
StemGameFunctionalTest.java	Added Selenium tests for login and page flow.	Tests integrated application from tester point of view.

4. Unit Test Cases - Developer Point of View

Test Case	Description	Expected Result	Actual Result	Coverage Purpose
testTrueAdd	Adds 1 + 2	3.0	Passed	Valid addition path.
testAddNumber1Empty	Empty first addition input	null	Passed	Prevents error page from empty input.
testAddInvalidInput	Non-numeric addition input	null	Passed	Handles invalid value safely.
testTrueSubtraction	Subtracts 5 - 2	3.0	Passed	Valid subtraction path.

testSubtractionEmptyInput	Empty subtraction input	null	Passed	Boundary case for missing value.
testSubtractionInvalidInput	Non-numeric subtraction input	null	Passed	Prevents NumberFormatException.
testTrueMultiplication	Multiplies 2 * 3	6.0	Passed	Valid multiplication path for Q3.
testMultiplicationEmptyInput	Empty multiplication input	null	Passed	Missing input validation for Q3.
testMultiplicationInvalidInput	Non-numeric multiplication input	null	Passed	Invalid input validation for Q3.

5. Unit Test Coverage Justification

Coverage Area	Covered By	Why It Is Important
Valid addition	testTrueAdd	Confirms Q1 math operation works.
Valid subtraction	testTrueSubtraction	Confirms Q2 math operation works.
Valid multiplication	testTrueMultiplication	Confirms Q3 math operation works.
Empty input handling	Empty input tests	Stops the application from showing an error page for blank fields.
Invalid input handling	Invalid input tests	Ensures letters or symbols do not crash the service.

These tests were derived from valid and invalid equivalence classes for math input fields. They cover normal values, empty values and non-numeric values, which are the main causes of errors in this application.

6. Functional Test Cases - Testing Team Point of View

Selenium Test	Scenario	Expected Result	Actual Result	Integration Area Covered
testLoginSuccessMovesToQ1	Valid login credentials submitted	Browser navigates to /q1	To be confirmed with final screenshot	Login JSP -> RoutingServlet -> LoginService -> Q1 page.
testQ1CorrectAnswerMovesToQ2	Correct addition answer	Browser navigates to /q2	Passed in current run	Q1 form -> q1 POST -> MathQuestionService -> redirect.
testQ1WrongAnswerStaysOnQ1	Wrong addition answer	Browser remains on /q1	Passed in current run	Wrong answer handling and feedback.
testQ2CorrectAnswerMovesToQ3	Correct subtraction answer	Browser navigates to /q3	Passed in current run	Q2 form and routing to Q3.
testQ3WrongAnswerStaysOnQ3	Wrong multiplication answer	Browser remains on /q3	Passed in current run	Q3 wrong-answer handling.

Note: The login functional test depends on the exact DOB format submitted by Selenium and accepted by LoginService.java. The safer implementation accepts both 2004-01-01 and 01/01/2004 to avoid browser date-format differences.

7. Selenium Functional Testing Explanation

Selenium tests were used to simulate a real student using the STEM game through a browser. The tests opened the web pages, entered numbers, submitted forms and checked the resulting URL. This validates the integrated application instead of only the isolated math functions.

JUnit Assert was used to check page navigation, for example whether the current URL contains /q2 after a correct Q1 answer. This confirms that the page, routing method, math service and redirect behaviour all worked together.

8. Reflection: Testing 8.1P Compared to 7.1P

Testing 8.1P was more complex than 7.1P because the application includes multiple JSP pages and multiple Java endpoints. In 7.1P, the main testing focus was login authentication and whether the browser title changed to success or fail. In 8.1P, the test must also verify multi-page navigation, math answer checking, wrong-answer handling and invalid input handling.

Unit testing is still useful because MathQuestionService.java can be tested directly and quickly without launching a browser. However, unit tests alone cannot prove that the web pages, form field names, servlet mappings and redirect behaviour are correct. Selenium functional testing is necessary because it validates the full integrated flow from the browser to Java endpoints and back to the browser.

The biggest lesson from this task is that integrated web application testing must combine developer-side and tester-side perspectives. The developer-side tests make sure the math logic is safe and correct, while the tester-side Selenium tests make sure the STEM game works as a real user experience.

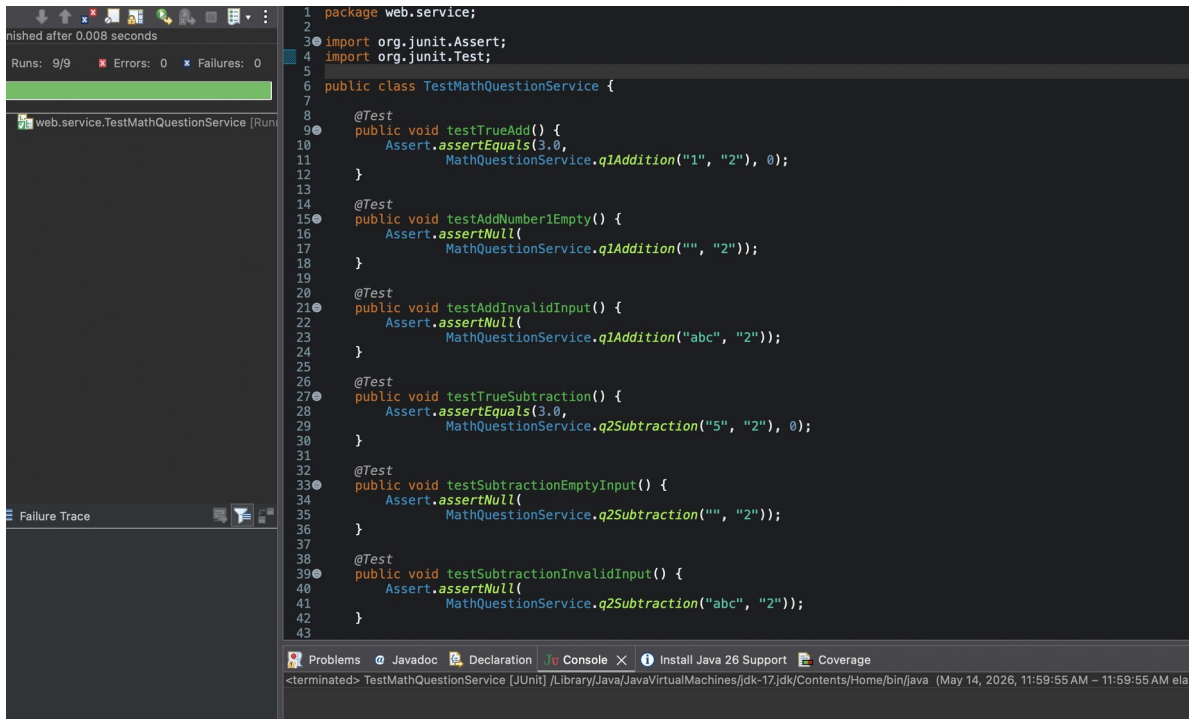
9. Testability of the Web Application Structure

Keeping JSP pages and Java endpoints inside one Spring Boot project improves testability because the complete application can be started locally through MyServer.java. The routing controller clearly maps each URL to a Java method, which makes it easier to design Selenium tests around expected URL changes such as /q1, /q2 and /q3.

However, integrated testing also creates extra challenges. Field ids and names in JSP pages must match Selenium locators and request parameter names. Any mismatch can cause Selenium NoSuchElementException or backend null values. This shows why both page inspection and backend validation are important.

10. Screenshots

Unit Test Success Screenshot - TestMathQuestionService.java



Selenium Functional Test Success Screenshot

[Insert final Selenium success screenshot here after all 5 tests pass.]

GitHub Repository Screenshot

[Insert GitHub repository screenshot here.]

11. Source Code

11.1 MathQuestionService.java

```

package web.service;

public class MathQuestionService {

    public static Double q1Addition(String number1, String number2) {
        try {
            if (number1 == null || number2 == null ||
                number1.trim().isEmpty() || number2.trim().isEmpty()) {
                return null;
            }
            return Double.valueOf(number1) + Double.valueOf(number2);
        } catch (NumberFormatException e) {
            return null;
        }
    }

    public static Double q2Subtraction(String number1, String number2) {
        try {
            if (number1 == null || number2 == null ||
                number1.trim().isEmpty() || number2.trim().isEmpty()) {
                return null;
            }
            return Double.valueOf(number1) - Double.valueOf(number2);
        } catch (NumberFormatException e) {

```

```

        return null;
    }
}

public static Double q3Multiplication(String number1, String number2) {
    try {
        if (number1 == null || number2 == null ||
            number1.trim().isEmpty() || number2.trim().isEmpty()) {
            return null;
        }
        return Double.valueOf(number1) * Double.valueOf(number2);
    } catch (NumberFormatException e) {
        return null;
    }
}
}

```

11.2 LoginService.java

```

package web.service;

public class LoginService {

    public static boolean login(String username, String password, String dob) {
        if (username == null || password == null || dob == null) {
            return false;
        }

        username = username.trim();
        password = password.trim();
        dob = dob.trim();

        return username.equals("ansh")
            && password.equals("ansh_pass")
            && (dob.equals("2004-01-01") || dob.equals("01/01/2004"));
    }
}

```

11.3 view-q3.jsp

```

<html>
<body>

<h2>Q3</h2>

<div>${message}</div><br/><br/>

<form action="/q3" method="post">
    <label for="number1">First number:</label><br>
    <input type="text" id="number1" name="number1"><br>

    <label for="number2">Second number:</label><br>
    <input type="text" id="number2" name="number2"><br>

    <label for="result">MULTIPLY:</label><br>
    <input type="text" id="result" name="result"><br><br>

    <input type="submit" value="Submit">
</form>

</body>
</html>

```

11.4 TestMathQuestionService.java

```
package web.service;

import org.junit.Assert;
import org.junit.Test;

public class TestMathQuestionService {

    @Test
    public void testTrueAdd() {
        Assert.assertEquals(3.0,
            MathQuestionService.q1Addition("1", "2"), 0);
    }

    @Test
    public void testAddNumber1Empty() {
        Assert.assertNull(MathQuestionService.q1Addition("", "2"));
    }

    @Test
    public void testAddInvalidInput() {
        Assert.assertNull(MathQuestionService.q1Addition("abc", "2"));
    }

    @Test
    public void testTrueSubtraction() {
        Assert.assertEquals(3.0,
            MathQuestionService.q2Subtraction("5", "2"), 0);
    }

    @Test
    public void testSubtractionEmptyInput() {
        Assert.assertNull(MathQuestionService.q2Subtraction("", "2"));
    }

    @Test
    public void testSubtractionInvalidInput() {
        Assert.assertNull(MathQuestionService.q2Subtraction("abc", "2"));
    }

    @Test
    public void testTrueMultiplication() {
        Assert.assertEquals(6.0,
            MathQuestionService.q3Multiplication("2", "3"), 0);
    }

    @Test
    public void testMultiplicationEmptyInput() {
        Assert.assertNull(MathQuestionService.q3Multiplication("", "3"));
    }

    @Test
    public void testMultiplicationInvalidInput() {
        Assert.assertNull(MathQuestionService.q3Multiplication("abc", "3"));
    }
}
```

11.5 StemGameFunctionalTest.java

```
package web.service;

import org.junit.Assert;
import org.junit.Test;
import org.openqa.selenium.By;
```

```

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class StemGameFunctionalTest {

    private void sleep(long sec) {
        try {
            Thread.sleep(sec * 1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    private WebDriver setupDriver() {
        System.setProperty(
            "webdriver.chrome.driver",
            "/Users/anshsharma1700/Documents/selenium/chromedriver"
        );
        WebDriver driver = new ChromeDriver();
        driver.manage().window().maximize();
        return driver;
    }

    private void login(WebDriver driver) {
        driver.navigate().to("http://127.0.0.1:8080/login");
        sleep(2);

        driver.findElement(By.id("username")).clear();
        driver.findElement(By.id("username")).sendKeys("ansh");

        driver.findElement(By.id("passwd")).clear();
        driver.findElement(By.id("passwd")).sendKeys("ansh_pass");

        driver.findElement(By.id("dob")).clear();
        driver.findElement(By.id("dob")).sendKeys("2004-01-01");

        driver.findElement(By.cssSelector("[type=submit]")).submit();
        sleep(2);
    }

    @Test
    public void testLoginSuccessMovesToQ1() {
        WebDriver driver = setupDriver();
        login(driver);
        Assert.assertTrue(driver.getCurrentUrl().contains("/q1"));
        driver.close();
    }

    @Test
    public void testQ1CorrectAnswerMovesToQ2() {
        WebDriver driver = setupDriver();
        driver.navigate().to("http://127.0.0.1:8080/q1");
        sleep(2);
        driver.findElement(By.id("number1")).sendKeys("1");
        driver.findElement(By.id("number2")).sendKeys("2");
        driver.findElement(By.id("result")).sendKeys("3");
        driver.findElement(By.cssSelector("[type=submit]")).submit();
        sleep(2);
        Assert.assertTrue(driver.getCurrentUrl().contains("/q2"));
        driver.close();
    }

    @Test
    public void testQ1WrongAnswerStaysOnQ1() {
        WebDriver driver = setupDriver();
        driver.navigate().to("http://127.0.0.1:8080/q1");
    }
}

```



```

        sleep(2);
        driver.findElement(By.id("number1")).sendKeys("1");
        driver.findElement(By.id("number2")).sendKeys("2");
        driver.findElement(By.id("result")).sendKeys("5");
        driver.findElement(By.cssSelector("[type=submit]")).submit();
        sleep(2);
        Assert.assertTrue(driver.getCurrentUrl().contains("/q1"));
        driver.close();
    }

    @Test
    public void testQ2CorrectAnswerMovesToQ3() {
        WebDriver driver = setupDriver();
        driver.navigate().to("http://127.0.0.1:8080/q2");
        sleep(2);
        driver.findElement(By.id("number1")).sendKeys("5");
        driver.findElement(By.id("number2")).sendKeys("2");
        driver.findElement(By.id("result")).sendKeys("3");
        driver.findElement(By.cssSelector("[type=submit]")).submit();
        sleep(2);
        Assert.assertTrue(driver.getCurrentUrl().contains("/q3"));
        driver.close();
    }

    @Test
    public void testQ3WrongAnswerStaysOnQ3() {
        WebDriver driver = setupDriver();
        driver.navigate().to("http://127.0.0.1:8080/q3");
        sleep(2);
        driver.findElement(By.id("number1")).sendKeys("2");
        driver.findElement(By.id("number2")).sendKeys("3");
        driver.findElement(By.id("result")).sendKeys("10");
        driver.findElement(By.cssSelector("[type=submit]")).submit();
        sleep(2);
        Assert.assertTrue(driver.getCurrentUrl().contains("/q3"));
        driver.close();
    }
}

```

12. Conclusion

This task successfully transformed the earlier login application into a multi-page STEM math game and tested it using both JUnit and Selenium. Unit tests verified the math service logic for valid, empty and invalid inputs. Selenium tests verified web page behaviour and navigation across login, Q1, Q2 and Q3 pages. The combined testing strategy improves confidence that the application works as an integrated web application rather than only as isolated Java functions.

13. References

Deakin University SIT333 Task 8.1P learning materials.

JUnit 4 Documentation. Available at: <https://junit.org/junit4/>

Selenium Documentation. Available at: <https://www.selenium.dev/documentation/>

Spring Boot Documentation. Available at: <https://spring.io/projects/spring-boot>